

Foundations of Machine Learning (AI2000/AI5000): (Assignment 3)

The “Mini-ML” Library and Autograd Engine

Released on: 23rd October, 2025

Deadline: 3rd November, 2025, EoD (11:59PM, IST)

Maximum Marks: 40M

A. Instructions

- Answer all questions. We encourage best coding practices. You may not get full marks if you make it difficult for us to understand your code. Hence, use intuitive names for variables and comment your code wherever necessary.
- Use markdown cells in your notebook(s) for briefly explaining the objective of a code cell or reporting observations as asked in the questions.
- It is expected that you work on these problems individually.
- If you have any doubts, please contact the TAs or the instructor no later than 2 days prior to the deadline.
- Use of LLMs is strictly prohibited. If found, you will be heavily penalized.
- You must build your models from scratch as specified. You may use built-in implementations only for basic functions (e.g., `sqrt`, `log`, `linalg.inv`, `numpy.random.multivariate_normal`) from libraries such as `numpy`. For plots, you may use `matplotlib`.
- You **must** use `numpy.random.seed()` for reproducibility.
- Students are required to submit a `.zip` file containing:
 1. Your Python package `my_ml_lib`.
 2. All Python scripts used for the experiments (e.g., `train_model.py`).
 3. A `saved_models/` directory containing the saved file(s) (`.pkl` for OvR, `.npz` for autograd models) for your **single best performing model** from Problem 5.
 4. A `README.md` file with instructions on how to reproduce the results

5. A **single PDF document** (preferably $\text{\LaTeX}$) containing all mathematical derivations and the final report/analysis for all problems.

B. Problems

This assignment is a capstone project designed to integrate everything you have learned. You will build a reusable ML library (like a mini-`scikit-learn`), implement classical models, and then build your own Neural Network framework (like a mini-PyTorch).

This would be useful for your whole journey in Artificial Intelligence. The vision is that each of you would be able to implement all the algorithms which you learn in this course, and future courses of Deep Learning, Natural Language Processing, Computer Vision and Natural Language Processing and would graduate with a mini library of yourselves.

Problem 1: The “Mini-ML” Library (3 Marks)

Your first task is to create a proper Python package that will house your ML algorithms. This will enforce a clean, reusable, and professional structure.

1. **Create the Library Structure:** Create a directory structure for your Python package, named `my_ml_lib`. This structure will house your old models from A2, your new models from A3 (Logistic Regression and the NN), and the new helper modules for preprocessing and datasets.

```
1  my_ml_lib/
2      __init__.py
3      datasets/
4          __init__.py
5          _loaders.py           # For spambase,
6  fashion_mnist # Required
7          _synthetic.py        # For noisy_sine (Optional
8
9      linear_models/
10         __init__.py
11         classification/
12             __init__.py
13             _lda.py           # Optional
14             _logistic.py      # For Problem 2
15             _least_squares.py # Optional
16             _perceptron.py    # Optional
17         regression/
18             __init__.py
19             _bayes.py         # Optional
20             _ridge.py         # Optional
21
22         model_selection/      Problem 3,5
23             __init__.py
24             _split.py         # train_test_val split
25             _kfold.py         # K Fold
26
27         naive_bayes/
28             __init__.py
29             _bnb.py           # Optional
30             _gnb.py           # Optional
```

```

30
31     preprocessing/                # Problem 3,5
32         __init__.py
33         _data.py                  # StandardScaler
34         _gaussian.py              # GaussianBasisFeatures
35         _polynomial.py            # PolynomialFeatures
36
37     nn/                            # Problem 4.5
38         __init__.py
39         autograd.py               # The Value class
40         modules/
41             __init__.py
42             base.py                # The base Module class
43             linear.py              # The Linear layer
44             activations.py         # ReLU, Sigmoid layers
45             containers.py          # The Sequential
46
47     container
48         optim.py                  # The SGD optimizer
49         losses.py                  # The CrossEntropyLoss

```

2. **Implement Required Modules & Models:** To manage the assignment scope (10 days), you are **only required** to implement or refactor the modules listed below. You must follow the `scikit-learn` interface for all classes.

- **Models** (e.g., class `MyModel`): Must have `__init__`, `fit(X, y)`, `predict(X)`, and (where applicable) `predict_proba(X)`.
- **Transformers** (e.g., class `MyTransformer`): Must have `__init__`, `fit(X, y=None)`, `transform(X)`, and `fit_transform(X, y=None)`.

Required Implementations:

- **In `my_ml_lib/preprocessing/`:** (New Modules)
 - `_polynomial.py`: Implement a class `PolynomialFeatures` transformer.
 - `_gaussian.py`: Implement a class `GaussianBasisFeatures` transformer (based on your A2 logic).
- **In `my_ml_lib/datasets/`:** (New Modules)
 - `_loaders.py`: Implement `def load_spambase()` and `def load_fashion_mnist()`. These functions should load data from local `.csv/` `.data` files.

Note: You are encouraged to refactor your other A1/A2 models (Ridge, Bayesian Reg, etc.) into this library for your own benefit, but it is not required for grading.

3. **Module Imports:** In each `__init__.py` file, you must import the classes/functions from their respective modules so they can be accessed by the user. For example, in `my_ml_lib/preprocessing/__init__.py`, you should have:

```

1     from ._polynomial import PolynomialFeatures
2     from ._gaussian import GaussianBasisFeatures
3
4     __all__ = [
5         'PolynomialFeatures',
6         'GaussianBasisFeatures'

```

This allows a user to write `from my_ml_lib.preprocessing import PolynomialFeatures`, which is the standard, clean interface. Do this for all new modules.

Problem 2: Logistic Regression (6 Marks)

Next, you will implement **L2-Regularized** Logistic Regression for binary classification and add it to your new library. You will use the **Iterative Reweighted Least Squares (IRLS)** algorithm, which is an application of the Newton-Raphson method.

The regularized loss function is: $L(\mathbf{w}) = -[\sum_{i=1}^N y_i \log(h_i) + (1 - y_i) \log(1 - h_i)] + \frac{\alpha}{2} \|\mathbf{w}\|^2$
The gradient (∇L) and Hessian (H) of this regularized loss are:

$$\nabla L(\mathbf{w}) = \mathbf{X}^T(\mathbf{h} - \mathbf{y}) + \alpha \mathbf{w}$$

$$H(\mathbf{w}) = \mathbf{X}^T \mathbf{R} \mathbf{X} + \alpha \mathbf{I}$$

where $h_i = \sigma(\mathbf{w}^T \mathbf{x}_i)$, $\mathbf{R}$ is a diagonal matrix with $R_{ii} = h_i(1 - h_i)$, and $\mathbf{I}$ is the identity matrix. The Newton-Raphson update rule is: $\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - H^{-1} \nabla L(\mathbf{w})$

1. **(Implementation)** Implement a class `LogisticRegression` in your `my_ml_lib/linear_models/classification/_logistic.py` file. (Note the new `_logistic.py` filename, following your library's convention).
2. This class must have the `scikit-learn` interface:
 - `__init__(self, alpha=0.0, max_iter=100, tol=1e-4)`: The `alpha` parameter controls the strength of the L2 regularization.
 - `fit(self, X, y)`: This method must implement the **regularized** IRLS (Newton-Raphson) algorithm to find the optimal $\mathbf{w}$. You should include a bias term (i.e., augment $\mathbf{X}$), but **do not regularize the bias term** (i.e., set the corresponding α component to 0 in the gradient and Hessian).
 - `predict_proba(self, X)`: Returns the predicted probabilities $\sigma(\mathbf{w}^T \mathbf{x})$.
 - `predict(self, X)`: Returns the final class labels (0 or 1) based on a 0.5 threshold.

Problem 3: Experiment - Spam Classification with Standardization (8 Marks)

Now, use your library to conduct a focused experiment on the **Spambase Dataset**. Your goal is to investigate the importance of feature scaling and hyperparameter tuning for your L2-Regularized Logistic Regression model.

1. **(Load Data)** Use your `load_spambase()` function from `my_ml_lib.datasets` to load the data. Split it into training and test sets (80/20)split.

Note: Validation set isn't required since dataset size is small, thus KFold Validation works good.

You may use the train test split function which you implement in problem 5

2. **(Implement Preprocessing)** Ensure you have implemented the `StandardScaler` class in `my_ml_lib/preprocessing/_data.py`. This class must have `fit(X)` (to compute μ and σ), `transform(X)` (to apply $z = (x - \mu) / \sigma$), and `fit_transform(X)`.
3. **(The Experiment)** You will train your `LogisticRegression` model (from Problem 2) on two versions of the training data:
 - (a) **Raw:** Use the original features without scaling.
 - (b) **Standardized:** Use your `StandardScaler` to standardize the features to 0 mean and unit variance. Remember to fit the scaler *only* on the training data and then transform both the training and test data.
4. **(Hyperparameter Tuning)** For **both** the raw and standardized versions, use your `my_ml_lib.model_selection.KFold` class to perform *Cross-Validation* (e.g., 5-fold) on the *training set* to find the best L2 regularization strength α for your `LogisticRegression` model. Test at least 5 values (e.g., $\alpha \in \{0.01, 0.1, 1, 10, 100\}$).
5. **(Analysis)** In your PDF report, create a section for this problem.
 - Report the **best** α found via cross-validation for both the raw and standardized data.
 - Report the final **training error** and **test error** (using the best α) for your `LogisticRegression` model on both the raw and standardized datasets. Present these results clearly, perhaps in a small table.
 - Write a brief observation discussing the impact of standardization. Did it improve performance? Did it change the best value of α ? Why is feature scaling often crucial when using regularized linear models?

Problem 4: The Autograd Engine & Visualization (13M)

This is the core of the assignment. You will build a mini-PyTorch, including the autograd engine and basic neural network layers.

Highly Recommended Resource: For understanding the concepts behind building an autograd engine like this, Andrej Karpathy's "micrograd" tutorial is an excellent resource: [Building micrograd](#).

1. **(The Engine - autograd.py)** Complete the `Value` class. This class wraps a `numpy` array and tracks operations to build a computation graph. You must implement or verify the `_backward` methods (or the logic within the operation definition that sets `_backward`) for the following core operations to ensure correct gradient propagation. Pay close attention to handling `numpy` array shapes and broadcasting correctly in the gradients.
 - Basic Arithmetic: `__add__`, `__mul__`, `__sub__`, `__neg__`, `__pow__` (scalar power), `__truediv__`
 - Matrix Multiplication: `__matmul__`
 - Activation / Elementary Functions: `relu`, `exp`, `log`
 - Reduction Operations: `sum`, `mean`

For Visualization: Ensure that when you create a new `Value` node during an operation, you store references to its parent `Value` nodes (e.g., in a `_prev` attribute) and the operation performed (e.g., in an `_op` attribute).

2. **(The Blueprints - `nn/modules/`)** Complete the `__call__` (forward pass) methods for the following layer classes. These classes inherit from the provided `Module` class in `base.py`.
 - In `linear.py`: Complete class `Linear(Module)`.
 - In `activations.py`: Complete class `ReLU(Module)` and class `Sigmoid(Module)`.
 - In `containers.py`: Complete class `Sequential(Module)`.
3. **(The Toolkit - `losses.py` & `optim.py`)** Complete the final pieces:
 - In `losses.py`: Complete class `CrossEntropyLoss(Module)` (`__call__` method).
 - In `optim.py`: Complete class `SGD` (`step` method).
4. **(Visualize the Graph)** To verify your autograd engine, you must visualize its computation graph. We provide a helper script (`visualize.py`) that uses the `graphviz` library.
 - (a) **(Develop Traversal)** Create a function `get_all_nodes_and_edges(root_node)` (perhaps inside `autograd.py` or a new utility file). This function starts from a final `Value` (e.g., `loss`) and traverses backward through the graph (using the stored parent references, e.g., `_prev`) to build a set of all unique `Value` nodes and a set of all edges (tuples representing parent $\rightarrow$ child connections) involved in the computation.
 - (b) **(Visualize)** Use the provided `visualize.py` helper function. Pass your sets of nodes and edges to it to generate a plot of the computation graph. Visualize the graph for a simple expression involving at least one matrix multiplication and one activation function (e.g., one forward pass of a tiny MLP like `Sequential(Linear(2, 3), ReLU())` on a single data point).
 - (c) **(Submission)** Include the generated graph `.png` in your final PDF report. Add a brief explanation (1-2 paragraphs) in your report describing the graph, verifying that the connections and operations shown match your expectations for the expression you visualized.

Problem 5: Finding the Best Generalizing Model (10 Marks)

Now, use your *entire* `my_ml_lib` (preprocessing transformers, classical models, and your new autograd engine) to find the best possible performance for different model architectures on the real-world **Fashion-MNIST** dataset.

(The Goal) Your objective is to find the configuration (hyperparameters, architecture) for each of the five specified model types that achieves the best **generalization performance**. You will use the **validation set** to guide your experimentation and select the best configuration, aiming to maximize accuracy on this set as a proxy for good generalization. The final performance will be measured on the held-out **test set**.

1. **(Implement Data Splitting)** First, ensure you have implemented the `train_test_val_split` function in your `my_ml_lib/model_selection/_split.py` module as required in Problem 1. This function should take your data (X, y) and split proportions (e.g., `train=0.7, val=0.1, test=0.2`) and return the corresponding six arrays.
2. **(Load and Split Data)** Load the original Fashion-MNIST **training data** (60k samples), normalize pixels (0-1). Use your `train_test_split` function to divide this data into a **training set** and a **validation set** (e.g., 80% train / 20% validation, or approximately 48k/12k samples). *The original 10k test set will not be used directly by you; a modified version will be used by the TAs for final evaluation.*
3. **(The Model Types)** You must investigate the following five approaches:
 - **1. OvR Logistic Regression :** Using your binary `LogisticRegression` (Problem 2) in a One-vs-Rest scheme.
 - **2. Softmax Regression (Raw) :** Using your autograd engine (`Sequential(Linear(...))`) directly on raw pixels.
 - **3. Softmax + Polynomial Features :** Applying `PolynomialFeatures` before your autograd Softmax Regression.
 - **4. Softmax + Gaussian Basis Features :** Applying `GaussianBasisFeatures` before your autograd Softmax Regression.
 - **5. MLP :** Using your autograd engine (`Sequential` with `Linear` and `ReLU` layers) directly on raw pixels.
4. **(The Task: Experiment and Tune)** For each of the 5 model types, you should experiment to find a configuration that maximizes test accuracy. Consider tuning:
 - **Regularization:** The L2 strength `alpha` (for OvR Logistic and potentially Softmax via manual implementation).
 - **Preprocessing Parameters:** The `degree` for Polynomial features, or `n_centers` and `sigma` for Gaussian features.
 - **Optimizer Settings (for autograd models):** Learning rate (`lr`), potentially batch size, number of epochs.
 - **MLP Architecture:** The number of hidden layers and the number of neurons in each hidden layer. Try different activation functions if you implemented them.

Document your experimentation process briefly. Show evidence of using the validation set to select your best configuration for each model type.
5. **(Standard Evaluation Practice)** Once you have selected the **best hyperparameters** for a model type based on the validation set, the standard practice is often to retrain that model configuration on the *combined* training and validation data. For this assignment, you can simply use the model trained on the training set with the hyperparameters selected via the validation set.
6. **(Final Evaluation on Test Set)** After finalizing your best configuration for each of the 5 model types using the training and validation sets, evaluate each of these

final models **once** on the held-out **test set** you created in step 2. This test set accuracy is your final reported performance metric.

7. **(Save Your Best Model)** Once you have selected your single best model configuration based on validation performance:

- If it's OvR Logistic: Save the K classifiers using `pickle`.
- If it's an Autograd Model: Use `save_state_dict` to save parameters to `.npz`.

Place these saved file(s) in a `saved_models/` directory for submission.

8. **(TA Evaluation on Hidden Test Set)** The TAs will load your submitted best model and evaluate its accuracy on a **hidden, potentially augmented version** of the Fashion-MNIST test set.

9. **(Submission Method - Your Choice)**

- **Option A (Standard - Jupyter Notebook):** Submit a `capstone_showdown.ipynb` file. This notebook should clearly show your experimentation process for each model type (e.g., trying different parameters), the code for your *best* configuration for each, and the generation of the final plot and results table.
- **Option B (CLI Script):** Submit `train_model.py` using `argparse`. Your script should be flexible enough to allow tuning via command-line arguments. Your report should show the commands used for your best runs.

Examples are given below

```
1  python train_model.py --model logistic --data-path-train
   data/fashion-mnist_train.csv --lr 0.01 --epochs 20
2
3
4
5  python train_model.py --model logistic --data-path-train
   data/fashion-mnist_train.csv --lr 0.01 --epochs 20
   --poly-degree 2
6
7
8
```

10. **(The Analysis)** Your PDF report must create a final section titled **Capstone Showdown Analysis**. This section is crucial and must contain:

- **A Plot:** A single plot showing the training loss curves (loss vs. training steps/epochs) for the **best performing configuration** you found for each of the autograd-based models: Softmax (Raw), Softmax (Poly), Softmax (RBF), and MLP.
- **A Table:** A results table showing the **best test accuracy (%)** achieved for all five model types after your tuning efforts.
- **A Discussion:** A short discussion (4-5 paragraphs) analyzing your results:
 - (a) Compare the peak performance achieved by OvR Logistic vs. Softmax (Raw). Which performed better after tuning, and why might that be?

- (b) Discuss the effectiveness of Polynomial and Gaussian Basis Features . How much did they improve upon the raw Softmax model? What challenges did you face when tuning models with these features?
 - (c) Compare the best accuracy achieved by any linear model (OvR or Softmax variations) to the best accuracy achieved by your MLP. Explain why the MLP was likely able to reach higher performance, referencing the **Universal Approximation Theorem** and the concept of learned features.
 - (d) Briefly describe the tuning process. What hyperparameters seemed most important for each model type? Were there any surprises?
-

A Note on Data Splitting (Train/Validation/Test):

In this problem, you loaded the original 60,000-sample Fashion-MNIST training data and split it into your own train, validation, and test sets. You then tuned hyperparameters using the validation set and reported final performance on your test set. **Why didn't we use the original 10,000-sample Fashion-MNIST test file?**

This train/validation/test split methodology mirrors standard practice in real-world machine learning and competitions like Kaggle. The final test set (like Kaggle's private leaderboard set, or data a model will see after deployment) must remain completely unseen during model development and hyperparameter tuning. If you tune your model based on the test set performance, you risk overfitting *specifically to that test set*, giving you an overly optimistic estimate of how well your model will generalize to new, unseen data.

The validation set acts as a proxy for the final test set during development. By finding hyperparameters that perform well on the validation set, you increase the chances that your model will also perform well on the truly unseen test set. The goal is to maximize performance on the *test set* by carefully tuning on the *validation set* to find a model that generalizes well, rather than one that simply memorizes the training data (overfitting).
